

Reflection: What OpenAI's Agents API Made Me Realize About My Career

Course: Agentic AI for Business and FinTech (SEEM5660)

Date: September 21, 2026

On September 10, OpenAI released the Agents API in public beta. I almost scrolled past it -- another API launch, another developer tool, not really relevant to a FinTech student. But then I read one line that stopped me: "The difficult part of production agents is increasingly everything around the model -- state, tools, retries, observability, and safe execution."

That sentence messed with my head, because three hours earlier I had been sitting in front of my homework, staring at a broken chain.

What Happened in My Homework

I was building a LangChain pipeline to read supermarket receipts. The idea is simple: send receipt images to DeepSeek's vision model, extract some numbers, add them up. I thought the hard part would be getting the model to read the receipts correctly. Turns out, that was the easy part.

The hard part was everything else. First, I tried using `with_structured_output` to force the model to return clean data. It crashed -- DeepSeek's thinking mode doesn't support `tool_choice`. So I switched to a prompt-based approach, asking the model to return JSON. That worked, but the model kept missing discount lines. Receipt 2 was off by a dollar. Receipt 7 was off by eleven dollars. I had to rewrite the prompt three times, add a `discount_lines` field so the model would list every discount it found, and write fallback parsing logic in case the response came back wrapped in markdown code fences.

By the time I got both answers correct -- HK\$1974.30 and HK\$2348.20 -- I realized something weird. The model didn't change between my failing and passing attempts. Same model, same temperature, same images. What changed was the stuff around it: the prompt, the parsing, the error handling, the self-checking logic. The chain. The harness.

And then I read OpenAI's announcement, and they were basically saying: yes, that's the point. The model is a component. The harness is the product.

Why This Bothered Me

I've been treating AI as a tool I'll use at work -- like Excel, or Python. Learn the tool, apply it, move on. My career plan was simple: graduate, get a job at a bank or a FinTech startup, use AI to do my work faster. The Agents API made me question whether that plan still makes sense.

If OpenAI is turning the orchestration layer into a managed API, then "person who knows how to call models" is not a career. It's a task. And it's a task that's being automated right now, by the very platforms being built this month. The question isn't whether I'll use AI at work. I will. The question is whether I'll be the person building the system, or the person the system replaces.

I don't mean to be dramatic. Models aren't going to replace FinTech professionals next year. But the direction is clear: value is moving from "I can call the API" to "I can design the system that calls the API reliably, safely, and in a way that produces auditable results." The first skill is becoming a commodity. The second isn't.

What I Actually Did in This Homework, and Why It Matters

Here's the thing that surprised me most: my homework was, in a very small way, exactly what OpenAI just productized. I didn't just call a model. I:

- * Managed state across seven receipts, encoding images and tracking which ones had been processed
- * Used `chain.batch()` to run them in parallel instead of one at a time
- * Handled a failure mode where the model's thinking mode broke structured output, and pivoted to a different approach
- * Wrote fallback JSON parsing for when the model wrapped its output in code fences
- * Added a `discount_lines` field so the model would self-audit by listing every discount before summing them
- * Dealt with the fact that the model missed discounts on two receipts, and had to figure out why

None of that is "using AI." All of that is "building the system around AI." And I didn't realize that's what I was doing until I read the Agents API announcement and recognized the pattern.

What Changes for Me

I'm not going to pretend this one API launch completely rewrote my career plan. But it did shift my thinking in a few concrete ways.

First, I'm going to stop treating frameworks like LangChain as "convenience wrappers." They're not. They're the layer where production value gets created. The chain I built for this homework -- with its parallel processing, error handling, and self-auditing prompt -- is a small version of what companies are paying people to build. I should take this more seriously, not just as homework but as practice for real work.

Second, I want to get better at the unglamorous parts: error handling, observability, failure analysis. When receipt 2 was off by a dollar, I had to dig in and figure out it was a missed discount line. That kind of debugging -- "why did the

chain produce the wrong answer, and where exactly did it go wrong?" -- feels like it'll be a real and valuable skill. It's not glamorous, but it's the work that decides whether an AI system can be trusted in production.

Third, I'm reconsidering what "FinTech + AI" actually means as a career. I used to think it meant "I work in finance and I use AI tools." Now I think it might mean "I build the AI systems that financial institutions rely on." Those are different jobs. The first one is getting automated. The second one is just getting started.

I don't have a fully formed plan yet. But I know I don't want to be the person who just calls the model. I want to be the person who builds the thing that makes the model useful.